

Publishable Paper Upgrade Plan

PUBLISHABLE_PAPER_UPGRADE_PLAN.md

This document is the revised full plan for turning the current benchmark work into a professional, submission-ready research paper while making explicit use of:

- 1 the strongest existing runs
- 2 the current methodology draft
- 3 the existing test suite
- 4 the current report-generation pipeline

It is intentionally grounded in the repository as it exists now, not in a hypothetical clean-slate paper.

1. Objective

The paper should make one clear scientific contribution:

A controlled, source-grounded dialogue-to-requirements pipeline can be benchmarked rigorously, and model rankings differ substantially between direct source-to-requirements generation and dialogue-mediated source recovery.

To support that claim professionally, the paper must do three things well:

- 1 present the methodology clearly and conservatively
- 2 use the strongest existing runs without overclaiming
- 3 demonstrate reproducibility through artifacts, scripts, and tests

2. External Basis for the Plan

This plan follows four classes of external guidance and applies them directly to this project.

2.1 Empirical Software Engineering Reporting

Kitchenham et al. argue that reporting guidance should specify what information belongs in which section and avoid duplication or ambiguous structure. Source: Evaluating Guidelines for Reporting Empirical Software Engineering Studies

Applied here:

- 1 separate research questions, methodology, results, and threats to validity
- 1 move implementation detail out of headline results unless it changes interpretation
- 1 use a fixed paper structure rather than project-log narration

2.2 Observed Reporting Failures in SE Experiments

Revoredo et al. show that many software engineering experiment papers omit required information, lack a standard reporting sequence, and often fail to anchor claims in clear experimental structure. Source: A Study into the Practice of Reporting Software Engineering Experiments

Applied here:

- 1 use explicit slice descriptions
- 1 state what each run is for

- 1 separate compact verified results from stress tests and broader comparative slices
- 1 add a reproducibility and artifact section as part of the paper, not only as repo documentation

2.3 Open Science and Artifact Expectations

ICSE 2025 states that artifact sharing should be the default and that papers should describe how artifacts are accessed. Source: ICSE 2025 Research Track Open Science Policy

Applied here:

- 1 build a paper artifact package, not just code
- 1 provide run IDs, configs, scripts, and outputs in a navigable structure
- 1 make the paper's reported numbers traceable to saved run artifacts

2.4 NLP Reproducibility and Benchmark Robustness

Magnusson et al. show that stronger reporting of metrics, infrastructure, and artifacts improves reproducibility. Source: Reproducibility in NLP: What Have We Learned from the Checklist?

Ailem et al. show that benchmark-level model comparisons can be sensitive to benchmark composition and aggregation assumptions. Source: Examining the Robustness of LLM Evaluation to the Distributional Assumptions of Benchmarks

Applied here:

- 1 report both aggregate and per-document results
- 1 state clearly what slice each result comes from
- 1 avoid relying on one metric or one document
- 1 keep deterministic semantic scoring as the backbone metric

3. Assets We Already Have

The paper should be built from existing assets instead of re-explaining the entire project from scratch.

3.1 Core Methodology Draft

Primary source:

- 1 project_workflow_and_results.md

This file already contains:

- 1 dialogue controller algorithm
- 1 proposition extraction algorithm
- 1 dual-layer evaluation design
- 1 hard-slice interpretation
- 1 4-document cross-model interpretation
- 1 key methodological cautions
- 1 source references that can be reused in the paper

3.2 Existing Paper-Facing Manuscript

Current manuscript source:

1 paper_manuscript_obsidian.md

This file is now the preferred narrative source, but it still needs more quantitative hardening and artifact integration.

3.3 Existing Report Artifacts

Primary cross-model report:

- 1 cross_model_comparison_report.pdf
- 1 cross_model_comparison_report.md
- 1 cross_model_comparison_summary.json

These should feed tables and appendix material rather than be treated as the paper itself.

3.4 Existing Reported Runs

The strongest currently useful runs are:

- 1 20260424T205611Z_pure_full
 - verified 2-document local benchmark - best compact local headline result
- 1 20260427T213602Z_pure_full
 - local hard-slice rerun - infrastructure-fix proof point
- 1 20260428T015358Z_pure_full
 - first frontier Gemini hard-slice comparison - useful for showing that model strength alone did not initially solve the pipeline
- 1 20260428T032711Z_pure_full
 - improved hard-slice Gemini rerun - strongest single-document frontier recovery result
- 1 20260429T001722Z_pure_full
 - Gemini 4-document paper slice
- 1 20260429T002305Z_pure_full
 - Ollama 4-document paper slice

3.5 Existing Test and Validation Assets

Primary test file:

- 1 tests/test_recall_pipeline.py

This suite already covers paper-relevant logic:

- 1 dialogue controller metadata exposure
- 2 support-unit splitting
- 3 contextual support-unit construction
- 4 semantic theme annotation
- 5 low-yield theme cap behavior
- 6 forced clarification logic
- 7 dialogue context scoping for evidence-bank extraction

- 8 batch construction rules
- 9 proposition deduplication
- 10 generic proposition detection
- 11 validation preference for claimed evidence turns
- 12 novelty gap selection
- 13 LLM candidate-pair generation
- 14 partial-credit matching logic
- 15 report model-metadata loading

Additional stability support already exists in:

- 1 run_pure_full_multi_seed.py

That script is especially important because it gives us a direct path to paper-quality variance reporting rather than single-run-only claims.

4. Paper Claim Architecture

The paper will be stronger if each claim is assigned to the run evidence best suited to support it.

4.1 Headline Claim

Recommended headline:

A controlled conversational pipeline can outperform a direct local baseline on a verified compact PURE slice, while a broader 4-document comparison shows that dialogue-mediated recovery and direct structured generation produce different cross-model rankings.

Why this is strongest:

- 1 it uses the clean verified 2-document result for the strongest positive claim
- 1 it uses the 4-document slice for broader comparison
- 1 it avoids claiming that the hard single-document frontier result is the sole headline

4.2 Supporting Claims

Claim A:

- 1 controlled dialogue can preserve enough information for strong source recovery
- 1 supported by 20260424T205611Z_pure_full

Claim B:

- 1 the hardest remaining problem is source-faithfulness, not unsupported hallucination
- 1 supported by 20260428T032711Z_pure_full and dialogue-grounding diagnostics

Claim C:

- 1 direct-source generation and dialogue-mediated recovery should be treated as different benchmark conditions
- 1 supported by the 4-document Gemini/Ollama comparison:
- 20260429T001722Z_pure_full - 20260429T002305Z_pure_full

4.3 Claims to Avoid

Do not claim:

- 1 full industrial generalization
- 1 that weighted LLM adjudication is equivalent to human validation
- 1 that the 4-document full-context comparison proves evidence-bank superiority or inferiority
- 1 that the hard-slice problem is fully solved

5. Full Use of Existing Runs

The current paper should explicitly reuse the run history as a structured narrative, not as scattered historical notes.

5.1 Run Roles in the Paper

Use each run for one job only:

| Run | Role in paper | | --- | --- | | 20260424T205611Z | compact verified positive headline | | 20260427T213602Z | local hard-slice stress baseline | | 20260428T015358Z | first frontier-model stress comparison | | 20260428T032711Z | improved hard-slice frontier rerun | | 20260429T001722Z | Gemini 4-document comparison arm | | 20260429T002305Z | Ollama 4-document comparison arm |

This prevents the manuscript from reading like an accumulated lab notebook.

5.2 Recommended Paper Result Layout

Main body:

- 1 Table: compact verified 2-document local result
- 2 Table: hard-slice single-document comparison
- 3 Table: 4-document aggregate cross-model result
- 4 Table: 4-document per-document recall breakdown

Appendix:

- 1 per-theme hard-slice coverage
- 2 weighted-vs-semantic disagreement audit
- 3 run configuration matrix
- 4 extended per-document diagnostics

6. Full Use of Existing Tests

The tests should not just remain engineering checks. They should support the paper's reproducibility story.

6.1 What the Current Tests Already Prove

The current suite supports the following paper-facing claims:

| Test area | Paper relevance | | --- | --- | | dialogue controller metadata | the reported controller version is explicit and stable | | forced clarification logic | the recall-improvement controller behavior is testable | | support-unit segmentation | the unit of analysis for dialogue coverage is reproducible | | evidence-bank context scoping | extraction context is not silently full-transcript by accident | | proposition deduplication | duplicate handling is not ad hoc | | generic proposition detection | rewrite triggering has a deterministic basis | | validator

candidate-pair generation | the secondary LLM judge uses a controlled shortlist | | partial-credit matching | weighted adjudication logic is explicit, not hand-waved | | report metadata loading | reported model/validator metadata is programmatically traceable |

6.2 How Tests Should Be Used in the Paper

The paper should not dump unit-test details into the main text. Instead:

- 1 mention in the artifact section that core benchmark logic is covered by regression tests
- 2 include the test categories in the artifact README
- 3 optionally add one appendix table mapping paper components to test coverage

6.3 Additional Tests Worth Adding

Before submission, add four more paper-facing tests:

- 1 **Cross-model summary consistency test**
 - verifies that aggregate report numbers equal the underlying per-document summaries
- 1 **Artifact-index completeness test**
 - verifies that all paper-reported run IDs have their required summary files
- 1 **Manuscript number consistency test**
 - checks that key values in the manuscript match the stored JSON summaries
- 1 **Report-generation smoke test**
 - ensures the report builders run successfully on the paper-report slice

These would materially strengthen the reproducibility story.

7. Professionalization Work Packages

Work Package A. Freeze the Scientific Narrative

Deliverables:

- 1 fixed abstract claim
- 1 fixed introduction claim
- 1 fixed conclusion claim

Action:

- 1 choose one compact headline result and one broader comparison result
- 1 remove any narrative drift between manuscript, draft, and report

Work Package B. Harden the Quantitative Section

Deliverables:

- 1 macro metrics where available
- 1 one stability subsection
- 1 one compact ablation subsection

Recommended implementation:

- 1 use run_pure_full_multi_seed.py on a small representative slice
- 1 report mean and standard deviation for recall and F1
- 1 add one table for:
 - no clarification vs clarification - full-context vs evidence-bank where comparable - semantic vs weighted interpretation

Work Package C. Build the Artifact Package

Deliverables:

- 1 ARTIFACT_README.md
- 1 artifact_index.json
- 1 paper run manifest
- 1 claim-to-run mapping
- 1 claim-to-test mapping

Recommended contents:

- 1 all run IDs cited in the manuscript
- 2 all report files cited in the manuscript
- 3 exact scripts used to produce them
- 4 environment assumptions
- 5 how to rerun or partially reproduce key results

Work Package D. Build the Evidence Appendix

Deliverables:

- 1 sample requirement-pair appendix
- 1 semantic vs weighted disagreement appendix
- 1 hard-slice theme appendix

Recommended scope:

- 1 20-30 examples
- 1 stratified by full, partial, disagreement, and miss

Work Package E. Final Venue Conversion

Deliverables:

- 1 BibTeX
- 1 static figures
- 1 final PDF draft
- 1 anonymization / disclosure pass if needed

8. Concrete Next Implementation Steps

The next high-value implementation sequence should be:

1 Create an artifact index

- machine-readable index of all runs cited in the manuscript

1 Add paper-facing reproducibility tests

- cross-model summary consistency - manuscript number consistency - report smoke test

1 Add a Stability and Ablation subsection

- use existing multi-seed support where possible

1 Add a claim-to-run and claim-to-test appendix

- this will make the paper feel much more disciplined

1 Only after that, do wording polish

- wording polish is much less valuable before the quantitative and artifact structure is locked

9. Recommended Immediate Deliverables

If we continue from here, the strongest next concrete outputs are:

- 1 data/docs/ARTIFACT_INDEX_PLAN.md or a direct artifact_index.json
- 2 a manuscript section 6.5 Stability and Ablation
- 3 new tests for report/manuscript consistency
- 4 an appendix-ready audit table for semantic vs weighted disagreements

Those four changes would make the paper substantially more professional than another round of prose-only editing.